

Name: Manasi Mahaveer Upadhye.

Roll No: 06

LAB EXERCISE: 1

Q. Demonstration on basic table operations.

creation of table, alter table, renaming tables, truncate table, drop table, describe table

Creation of table:

EXAMPLE:

create table employee(emp_no number(5), emp_name varchar(20), job char(20), dep_no number(3));

SQL> desc employee;

```
SQL> desc employee;
```

Name	Null?	Type
EMP_NO		NUMBER(5)
EMP_NAME		VARCHAR2(20)
JOB		CHAR(20)
DEP_NO		NUMBER(3)

Alter Table

a) To Add column to existing Table

Example:

SQL> alter table employee add (phone_no char(10));

SQL> desc employee;

```
SQL> desc employee;
```

Name	Null?	Type
EMP_NO		NUMBER(5)
EMP_NAME		VARCHAR2(20)
JOB		CHAR(20)
DEP_NO		NUMBER(3)
PHONE_NO		CHAR(10)

b) To Add Multiple columns to existing Table

Example:

SQL> alter table employee add(salary number(7), age number(5));

SQL> desc employee;

```
SQL> desc employee;
```

Name	Null?	Type
EMP_NO		NUMBER(5)
EMP_NAME		VARCHAR2(20)
JOB		CHAR(20)
DEP_NO		NUMBER(3)
PHONE_NO		CHAR(10)
SALARY		NUMBER(7)
AGE		NUMBER(5)

c) Modifying Existing Columns**Example:**

```
SQL> alter table employee modify(emp_name varchar(30));
```

```
SQL> desc employee;
```

```
SQL> desc employee;
```

Name	Null?	Type
EMP_NO		NUMBER(5)
EMP_NAME		VARCHAR2(30)
JOB		CHAR(20)
DEP_NO		NUMBER(3)
SALARY		NUMBER(7)
AGE		NUMBER(5)

d) To Rename a column**Example:**

```
SQL> rename employee to employee1;
```

```
SQL> desc employee1;
```

```
SQL> desc employee1;
```

Name	Null?	Type
EMP_NO		NUMBER(5)
E_NAME		VARCHAR2(30)
JOB		CHAR(20)
DEP_NO		NUMBER(3)
PHONE_NO		CHAR(10)
SALARY		NUMBER(7)
AGE		NUMBER(5)

e) Truncate Table:

```
SQL> truncate table employee1;
```

```
SQL> truncate table employee1;
```

```
Table truncated.
```

```
SQL> desc employee1;
```

Name	Null?	Type
EMP_NO		NUMBER(5)
E_NAME		VARCHAR2(30)
JOB		CHAR(20)
DEP_NO		NUMBER(3)
PHONE_NO		CHAR(10)
SALARY		NUMBER(7)
AGE		NUMBER(5)

f) Dropping Table**EXAMPLE:**

```
SQL> drop table employee1;
```

```
SQL> desc employee;
```

```
SQL> commit;
```

```
Commit complete.
```

```
SQL> drop table employee1;
```

```
Table dropped.
```

LAB EXERCISE: 2**Q. Demonstration to Create an Employees table and perform different operations on table.**

- Add a column commission with domain to the Employee table.
- Insert any five records into the table.
- Update the column details of job
- Rename the column of Employ table using alter command.
- Delete the employee whose Empno is 105.

SQL> create table employees(emp_no number, emp_name varchar2(10), job varchar2(30), mgr number, salary number);

```
SQL> desc employees;
```

Name	Null?	Type
EMPNO		NUMBER
ENAME		VARCHAR2(10)
JOB		VARCHAR2(10)
MGR		NUMBER
SAL		NUMBER

a) Add a column commission with domain to the Employee table.

SQL> alter table employee add(commission number);

SQL> desc employees;

```
SQL> desc employees;
```

Name	Null?	Type
EMPNO		NUMBER
ENAME		VARCHAR2(10)
JOB		VARCHAR2(10)
MANAGER_NO		NUMBER
SAL		NUMBER
COMISSION		NUMBER

b) Insert any five records into the table.

SQL> insert into employees values(101,'Manasi','Web Dev',23,35000,3);

1 row created.

SQL> insert into employees values(102,'Vaishnavi','Data Ana',23,25000,5);

1 row created.

SQL> insert into employees values(103,'Saloni','Manager',24,25000,10);

1 row created.

SQL> insert into employees values(104,'Paras','Designer',27,26000,11);

1 row created.

SQL> insert into employees values(105,'Priyanka','Designer',24,21000,13);

1 row created.

SQL> select * from employee;

EMPNO	ENAME	JOB	MGR	SAL	COMISSION
101	Manasi	Web Dev	23	35000	3
102	Vaishnavi	Data Ana	23	25000	5
103	Saloni	Manager	24	25000	10
104	Paras	Designer	27	26000	11
105	Priyanka	Designer	24	21000	13

c) Update the column details of job

SQL> update employee set Job='Trainee' where emp_no=103;

SQL> select * from employees;

SQL> select * from employees;

EMPNO	ENAME	JOB	MGR	SAL	COMISSION
101	Manasi	Web Dev	23	35000	3
102	Vaishnavi	Data Ana	23	25000	5
103	Saloni	Trainee	24	25000	10
104	Paras	Designer	27	26000	11
105	Priyanka	Designer	24	21000	13

d) Rename the column of Employee table using alter command.

SQL> alter table employee rename column mgr to manager_no;

Table altered.

SQL> select * from employees;

SQL> desc employees;

Name	Null?	Type
EMPNO		NUMBER
ENAME		VARCHAR2(10)
JOB		VARCHAR2(10)
MANAGER_NO		NUMBER
SAL		NUMBER
COMISSION		NUMBER

e)Delete the employee whose Empno is 105.

SQL> delete employee where emp_no=105;

1 row deleted

SQL> select * from employees;

SQL> select * from employees;

EMPNO	ENAME	JOB	MANAGER_NO	SAL	COMISSION
101	Manasi	Web Dev	23	35000	3
102	Vaishnavi	Data Ana	23	25000	5
103	Saloni	Trainee	24	25000	10
104	Paras	Designer	27	26000	11

LAB EXERCISE: 3

Q. Demonstration Create a Department table and perform different operations on table. Create department table with the following structure.

SQL> create table depart(dept_no number, dept_name varchar2(10), location varchar2(10));

Table created.

SQL> desc depart

```
SQL> desc depart
Name                               Null?    Type
-----
DEPT_NO                            NUMBER
DEPT_NAME                          VARCHAR2(10)
LOCATION                             VARCHAR2(10)
```

a)Add column designation to the department table.

SQL> alter table depart add(designation varchar(10));

SQL> desc depart

```
SQL> desc depart
Name                               Null?    Type
-----
DEPT_NO                            NUMBER
DEPT_NAME                          VARCHAR2(10)
LOCATION                             VARCHAR2(10)
DESIGNATION                        VARCHAR2(10)
```

b)Insert values into the table.

SQL> insert into depart values(&dept_no, '&dept_name', '&location', '&designation');

Enter value for dept_no: 9

Enter value for dept_name: accounting

Enter value for location: Mumbai

Enter value for designation: manager

SQL> insert into depart values(&dept_no, '&dept_name', '&location', '&designation');

Enter value for dept_no: 10

Enter value for dept_name: IT

Enter value for location: Sangli

Enter value for designation: analyst

SQL> insert into depart values(&dept_no, '&dept_name', '&location', '&designation');

Enter value for dept_no: 11

Enter value for dept_name: hr

Enter value for location: Pune

Enter value for designation: HR manager

SQL> insert into depart values(&dept_no, '&dept_name', '&location', '&designation');

Enter value for dept_no: 12

Enter value for dept_name: design

Enter value for location: indore

Enter value for designation: UI lead

SQL> insert into depart values(&dept_no, '&dept_name', '&location', '&designation');

Enter value for dept_no: 13

Enter value for dept_name: research

Enter value for location: nagpur

Enter value for designation: r&d head

SQL> select * from depart;

```
SQL> select * from depart;
```

DEPT_NO	DEPT_NAME	LOCATION	DESIGNATIO
9	accounting	mumbai	manager
10	IT	Sangli	analyst
11	hr	Pune	Hr manager
12	design	indore	Ui lead
13	research	nagpur	r&d head

c)List the records of dept table grouped by dept_no.

SQL> select dept_no,dept_name from depart group by dept_no,dept_name;

```
SQL> select dept_no,dept_name from depart group by dept_no,dept_name;
```

DEPT_NO	DEPT_NAME
9	accounting
11	hr
12	design
10	IT
13	research

d)Update the record where dept no is 9.

SQL> update depart set designation='accounting' where dept_no=9;

SQL>select * from depart;

```
SQL> select * from depart;
```

DEPT_NO	DEPT_NAME	LOCATION	DESIGNATIO
9	accounting	mumbai	accounting
10	IT	Sangli	analyst
11	hr	Pune	Hr manager
12	design	indore	Ui lead
13	research	nagpur	r&d head

e)Delete any column data from the table.

SQL> alter table depart drop(designation);

SQL>select * from depart;

```
SQL> select * from depart;
```

DEPT_NO	DEPT_NAME	LOCATION
9	accounting	mumbai
10	IT	Sangli
11	hr	Pune
12	design	indore
13	research	nagpur

LAB EXERCISE: 4

Q. Demonstration on Queries using DDL, DML and TCL.

- a. Create a user and grant all permissions to the user.

```
SQL> connect
Enter user-name: system
Enter password:
Connected.
```

- b. Create user Manasi06 identified by CIMDR;

```
SQL> CREATE USER Manasi06 IDENTIFIED BY CIMDR;
User created.
```

- c. Create a user and grant

```
SQL> GRANT CONNECT, RESOURCE TO Manasi06;
Grant succeeded.
```

- d. Insert the any three records in the emp4 table and use rollback check the result

SQL> insert into emp4 values(101,'Samiksha',30000);

```
SQL> insert into emp4 values(101,'Samiksha',30000);
1 row created.
```

SQL> insert into emp4 values(102,'Shreya',40000);

```
SQL> insert into emp4 values(102,'Shreya',40000);
1 row created.
```

SQL> insert into emp4 values(103,'Veena',35000);

```
SQL> insert into emp4 values(103,'Veena',35000);
1 row created.
```

SQL> select * from emp4;

```
SQL> select * from emp4;

EMP_ID ENAME          SALARY
-----
101 Samiksha          30000
102 Shreya           40000
103 Veena            35000
```



```
SQL> rollback;
```

```
SQL> rollback;
```

```
Rollback complete.
```

e. Add primary key constraint and not null constraint to the emp4 table.

```
SQL> alter table emp4 add constraint emp_pk primary key (emp_id);
```

```
SQL> alter table emp4 add constraint emp_pk primary key(emp_id);
```

```
Table altered.
```

```
SQL> alter table employee modify ename not null;
```

```
SQL> alter table emp4 modify ename not null;
```

```
Table altered.
```

```
SQL> insert into emp4 values(104,null,40000);
```

```
SQL> insert into emp4 values(104,null,40000);
```

```
insert into emp4 values(104,null,40000)
```

```
*
```

```
ERROR at line 1:
```

```
ORA-01400: cannot insert NULL into ("SAMIKSHA"."EMP4"."ENAME")
```

```
SQL> insert into emp4 values(null,'Vaishnavi',40000);
```

```
SQL> insert into emp4 values(null,'Vaishnavi',40000);
```

```
insert into emp4 values(null,'Vaishnavi',40000)
```

```
*
```

```
ERROR at line 1:
```

```
ORA-01400: cannot insert NULL into ("SAMIKSHA"."EMP4"."EMP_ID")
```

LAB EXERCISE: 5**Q. Demonstration on Queries using Aggregate function**

COUNT,AVG,MIN,MAX,SUM,Group by and order by having

a.Create a table**(i) Create Employee table containing all Records.**

```
SQL> create table emp(Empid number(4),Empname varchar2(10),age
number,salary number);
```

```
SQL> select * from emp;
```

```
SQL> desc emp;
Name                               Null?    Type
-----
EMPID                               NUMBER(4)
EMPNAME                             VARCHAR2(10)
AGE                                  NUMBER
SALARY                               NUMBER

SQL> select * from emp;

   EMPID EMPNAME      AGE      SALARY
-----
    101  Anu         22       9000
    102  Shane        29       8000
    103  Rohan        34       6000
    104  Scott        44      10000
    105  Tiger        35        800
    106  Alex         27       7000
    107  Abhi         29        800

7 rows selected.
```

(ii)Count number of employee names from employee table.

```
SQL> select count(Empname) from emp;
```

```
SQL> select count(Empname) from emp;

COUNT(EMPNAME)
-----
                7
```

(iii)Find the Maximum age from employee table.

```
SQL> select max(age) from emp;
```

```
SQL> select max(age)from emp;

MAX(AGE)
-----
       44
```

(iv)Find the Minimum age from employee table.

```
SQL> select min(age) from emp ;
```

```
SQL> select min(age)from emp;

MIN(AGE)
-----
       22
```

Display the Sum of age employee

SQL> select sum(age)from emp;

```
SQL> select sum(age)from emp;

SUM(AGE)
-----
      220
```

Display the Average of age from Employee table.

SQL> select avg(age) from emp;

```
SQL> select avg(age)from emp;

AVG(AGE)
-----
31.4285714
```

Create a View for age in employee table.

SQL> create or replace view A as select age from emp where age<30;

View created.

Displayviews

SQL> select * from A;

```
SQL> select * from A;

AGE
-----
   22
   29
   27
   29
```

(v) Find grouped salaries of employees.(group by clause)

SQL> select salary from emp group by salary;

```
SQL> select salary from emp group by salary;

SALARY
-----
   9000
  10000
   8000
   6000
    800
   7000
```

(x).Find salaries of employee in Ascending Order.(order by clause)

SQL> select Empname,salary from emp order by salary;

```
SQL> select Empname,salary from emp order by salary;

EMPNAME      SALARY
-----
Abhi          800
Tiger         800
Rohan        6000
Alex         7000
Shane        8000
Anu          9000
Scott       10000
```

(xi) Find salaries of employee in DescendingOrder.

SQL> select Empname,salary from emp order by salary desc;

```
SQL> select Empname,salary from emp order by salary desc;
```

EMPNAME	SALARY
-----	-----
Scott	10000
Anu	9000
Shane	8000
Alex	7000
Rohan	6000
Abhi	800
Tiger	800

(xii) HavingClause.

SQL> select Empname,salary from emp where age<29 group by Empname,salary having salary<10000;

```
SQL> select Empname,salary from emp where age<29 group by Empname,salary having salary<10000;
```

EMPNAME	SALARY
-----	-----
Alex	7000
Anu	9000

LAB EXERCISE: 6

Q. Demonstration Program on PL/SQL and exception handling

1. Write a pl/sql block to find the maximum number from given three number

SQL> set serveroutput on;

SQL> declare

```
2   a number;
3   b number;
4   c number;
5   begin
6   a:=&a;
7   b:=&b;
8   c:=&c;
9   if(a>b and a>c) then
10  dbms_output.put_line('a is maximum'||a);
11  elsif(b>a and b>c)then
12  dbms_output.put_line('b is maximum'||b);
13  else
14  dbms_output.put_line('c is maximum'||c);
15  end if;
16  end;
17  /
```

OUTPUT:



```
Enter value for a: 10
old 6: a:=&a;
new 6: a:=10;
Enter value for b: 20
old 7: b:=&b;
new 7: b:=20;
Enter value for c: 30
old 8: c:=&c;
new 8: c:=30;
c is maximum 30

PL/SQL procedure successfully completed.

SQL> |
```

2. Write a PL/SQL program for swapping two number

SQL> declare

```
2   a number(3);
3   b number(3);
4   begin
5   a:=&a;
6   b:=&b;
7   dbms_output.put_line('Before Swaping a='||a||' and b='||b);
8   a:=a+b;
9   b:=a-b;
10  a:=a-b;
11  dbms_output.put_line('After Swaping a='||a||'and b='||b);
12  end;
13  /
```

Output:

```
Enter value for a: 10
old 5: a:=&a;
new 5: a:=10;
Enter value for b: 20
old 6: b:=&b;
new 6: b:=20;
Before Swaping a=10 and b=20
After Swaping a=20and b=10

PL/SQL procedure successfully completed.

SQL> |
```

3. Write a PL/SQL program to find the total of 4 subject and display the grade

SQL> declare

```
2   java number(10);
3   db number(10);
4   os number(10);
5   fintech number(10);
6   total number(10);
7   avgOfMarks number(20);
8   begin
9   dbms_output.put_line('Enter The marks of 4 subject');
10  java:=&java;
11  db:=&db;
12  os:=&os;
13  fintech:=&fintech;
14  total:=(java+db+os+fintech);
15  avgOfMarks:=(total/400)*100;
16  if java<40 or db<40 or os<40 or fintech<40 then
17  dbms_output.put_line('Fail');
18  end if;
19  if avgOfMarks>75 then
20  dbms_output.put_line('Grade A');
21  elsif avgOfMarks>65 and avgOfMarks<75 then 22
22  dbms_output.put_line('Grade B');
23  elsif avgOfMarks>55 and avgOfMarks<65 then
24  dbms_output.put_line('Grade C');
25  else
26  dbms_output.put_line('Invalid Input');
27  end if;
28  dbms_output.put_line('Percentage is '||avgOfMarks);
29  end;
30  /
```

OUTPUT:

```
Enter value for java: 80
old 10:      java:=&java;
new 10:      java:=80;
Enter value for db: 70
old 11:      db:=&db;
new 11:      db:=70;
Enter value for os: 75
old 12:      os:=&os;
new 12:      os:=75;
Enter value for fintech: 90
old 13:      fintech:=&fintech;
new 13:      fintech:=90;
Enter The marks of 4 subject
Grade A
Percentage is 79
PL/SQL procedure successfully completed.
```

4. Write a program to accept a number and find sum of digit.

```
SQL> declare
2   n number(5):=&n;
3   s number:=0;
4   r number:=0;
5   begin
6   while n!=0 loop
7   r:=mod(n,10);
8   s:=s+r;
9   n:=trunc(n/10);
10  end loop;
11  dbms_output.put_line('Sum of digits of given number is '||s);
12  end;
13  /
```

Output:

```
Enter value for n: 123
old 2: n number(5):=&n;
new 2: n number(5):=123;
Sum of digits of given number is 6
PL/SQL procedure successfully completed.
SQL> |
```

5. PL/SQL Program to accept a number from user and print number in reverse order.

```
SQL> declare
2   num number(10):=&num;
3   rev number(10):=0;
4   m number(10):=0;
5   begin
6   while num!=0 loop
7   m:=mod(num,10);
8   rev:=rev*10+m;
9   num:=floor(num/10);
10  end loop;
11  dbms_output.put_line('Reverse number is '||rev);
12  end; 13 /
```

Output:

```
Enter value for num: 187
old 2: num number(10):=&num;
new 2: num number(10):=187;
Reverse number is 781

PL/SQL procedure successfully completed.

SQL> |
```

6. Write a PL / SQL program to check whether the given number is prime or not

SQL> declare

```
2   num number;
3   i number:=1;
4   c number:=0;
5   begin
6   num:=&num;
7   for i in 1..num loop
8   if (mod(num,i)=0) then
9   c:=c+1;
10  end if;
11  end loop;
12  if c>2 then
13  dbms_output.put_line(num||' not a prime number');
14  else
15  dbms_output.put_line(num|| 'is a prime number');
16  end if;
17  end;
18  /
```

Output:

```
Enter value for num: 5
old 6:      num:=&num;
new 6:      num:=5;
5is a prime number

PL/SQL procedure successfully completed.

SQL> |
```

7. Write a PL/SQL program to find the factorial of a given number.

SQL> declare

```
2   i number(4):=1;
3   f number(10):=1;
4   num number(10);
5   begin
6   num:=&num;
7   for i in 1..num loop
8   f:=f*i;
9   end loop;
10  dbms_output.put_line('Factorial of '||num||' is '||f);
11  end;
12  /
```


Output:

```
Enter value for num: 5
old 6: num:=&num;
new 6: num:=5;
Factorial of 5 is 120

PL/SQL procedure successfully completed.

SQL> |
```

8. calculate the area of a circle for a value of radius varying from 3 to 7. Store the radius and the corresponding values of calculated area in table areas.

Consisting of two columns radius and area .

```
SQL> create table area(radius number,area number);

Table created.

SQL> |
```

```
SQL> set serveroutput on;
SQL> declare
2  pi constant number(4,2):=3.14;
3  radius number(5);
4  area number(14,2);
5  begin
6  radius:=3;
7  while radius<=7 loop
8  area:=pi*power(radius,2);
9  insert into area values(radius,area);
10 radius:=radius+1;
11 end loop;
12 end;
13 /
```

Output:

```
SQL> select * from area;
```

```
PL/SQL procedure successfully completed.

SQL> select * from area;

  RADIUS      AREA
-----
        3      28.26
        4      50.24
        5      78.5
        6     113.04
        7     153.86

SQL> |
```

9. Write a PL/SQL program to accept a string and remove the vowels from the string. (When 'hello' passed to the program it should display 'Hll' removing e and o from the world Hello)

```
SQL> set serveroutput on
SQL> set verify off
SQL> Accept v_string PROMPT 'Enter String';
Enter String Hello
SQL> declare
2 v_new_string varchar2(100);
3 begin
4 v_new_string:=regexp_replace('&v_string','[aeiouAEIOU]','');
5 DBMS_output.put_line('New String will be '||v_new_string);
6 end;
7 /
```

New String will be Hll

PL/SQL procedure successfully completed.

Output

```
SQL> declare
2 v_new_string varchar2(100);
3 begin
4 v_new_string:=regexp_replace('&v_string','[aeiouAEIOU]','');
5 DBMS_output.put_line('New String will be '||v_new_string);
6 end;
7 /
New String will be Hll

PL/SQL procedure successfully completed.
```

10. Write a PL/SQL program to accept a number and a divisor. Make sure the divisor is less than or equal to 10. Else display an error message.

Otherwise Display the remainder

```
SQL> declare
2 num number;
3 div number;
4 result number;
5 begin
6 num:=&num; 7 div:=&div;
8 if div>10 then
9 dbms_output.put_line('Error:div should be less or equal
to 10'); 10 else
11 result:=num/div;
12 dbms_output.put_line('Division will be '||result);
13 end if;
14 end;
15 /
```

OUTPUT

```
Enter value for num: 30
Enter value for div: 10
Division will be 3

PL/SQL procedure successfully completed.

SQL> |
```

LAB EXERCISE: 7**Q. Demonstration Procedures and functions.**

1) calculate the net salary and year salary if da is 30% of basic, hra is 10% of basic and pf is 7% if basic salary is less than 8000, pf is 10% if basic sal between 8000 to160000

```
SQL> declare
2     ename varchar(20);
3     basic number;
4     da number;
5     hra number;
6     pf number;
7     netsalary number;
8     year number;
9     begin
10    ename:='&ename';
11    basic:=&basic;
12    da:=basic*(30/100);
13    hra:=basic*(10/100);
14    if basic<80000 then
15    pf:=basic*(8/100);
16    elsif basic>=80000 and basic<=160000 then
17    pf:=basic*(10/100);
18    end if;
19    netsalary:=(basic+da+hra)-pf;
20    year:=netsalary*12;
21    dbms_output.put_line('Employee name '||ename);
22    dbms_output.put_line('Providend Fund '||pf);
23    dbms_output.put_line('Net Salary '||netsalary);
24    dbms_output.put_line('Year Salary '||year);
25    end;
26    /
```

Output:

```
Enter value for ename: Veena
Enter value for basic: 25000
Employee name Veena
Providend Fund 2000
Net Salary 33000
Year Salary 396000

PL/SQL procedure successfully completed.

SQL> |
```

2) Create a function on to find the factorial of a given number and hence find NCR

SQL> create or replace func on fact(n number) return number is

```

2      a number:=n;
3      f number:=1;
4      i number;
5      begin
6      for i in 1 ..n loop
7      f:=f*a;
8      a:=a-a;
9      end loop;
10     return f;
11     end;
12     /

```

Output:

```

SQL> select fact(5) from dual;

      FACT(5)
-----
          120

```

Function on To find NCR

SQL> create or replace func on ncr(n number,r number) return number is

```

2  n1 number:=fact(n);
3  r1 number:=fact(r);
4  nr1 number:=fact(n-r);
5  result number;
6  begin
7  result:=(n1)/(r1*nr1);
8  return result;
9  end; 10 /

```

Output:

```

SQL> select ncr(5,5) from dual;

      NCR(5,5)
-----
              1

```

3) Print Fibonacci series using local function.

SQL> create or replace func on fib (n posi ve) return integer is

```

2  begin
3  if (n=1) or (n=2) then
4  return 1;
5  else
6  return fib(n-1)+fib(n-2);

```

```

7  end if;
8  end;
9  /

```

Output:

```

SQL> select fib(5) from dual;

      FIB(5)
-----
          5

```

4) write a pl/sql function on accept date of birth as "dd-mm-yyyy" and sum all digits till you get single digit number to show as he lucky number

```

SQL> declare
2  I_input varchar(20):='31/01/1978';
3  I_output int;
4  begin
5  loop
6  dbms_output.put_line("");
7  dbms_output.put_line('I_input '||I_input);
8  I_output:=0;
9  for i in 1..length(I_input)loop
10 if substr(I_input,i,1)between '0' and '9' then
11 I_output:=I_output+to_number(substr(I_input,i,1));
12 end if;
13 end loop;
14 dbms_output.put_line('I_output='||I_output);
15 exit when I_output<10;
16 I_input:=to_char(I_output);
17 end loop;
18 dbms_output.put_line("");
19 dbms_output.put_line('Lucky ='||I_output);
20 end;
21 /

```

Output:

```

I_input 31/01/1978
I_output=30
I_input 30
I_output=3
Lucky =3

PL/SQL procedure successfully completed.
SQL> /

```

LAB EXERCISE: 8

Q. Demonstration on Trigger

1. Create a row level trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table.

This trigger will display the salary difference between the old values and new values

```
SQL> SET SERVEROUTPUT ON
```

```
SQL> CREATE OR REPLACE TRIGGER display_salary_changes
```

```
2  BEFORE INSERT OR UPDATE OR DELETE
```

```
3  ON customer
```

```
4  FOR EACH ROW
```

```
5  DECLARE
```

```
6  sal_diff NUMBER;
```

```
7  BEGIN
```

```
8  -- INSERT
```

```
9  IF INSERTING THEN
```

```
10 DBMS_OUTPUT.PUT_LINE('New salary: ' || :NEW.salary); 11
```

```
12  -- DELETE
```

```
13  ELSIF DELETING THEN
```

```
14  DBMS_OUTPUT.PUT_LINE('Old salary: ' || :OLD.salary);
```

```
15
```

```
16  -- UPDATE
```

```
17  ELSIF UPDATING THEN
```

```
18  sal_diff := :NEW.salary - :OLD.salary;
```

```
19
```

```
20  DBMS_OUTPUT.PUT_LINE('Old salary: ' || :OLD.salary);
```

```
21  DBMS_OUTPUT.PUT_LINE('New salary: ' || :NEW.salary);
```

```
22  DBMS_OUTPUT.PUT_LINE('Salary difference: ' || sal_diff);
```

```
23  END IF;
```

```
24  END;
```

```
25  /
```

Trigger created.

OUTPUT:

```
SQL> INSERT INTO customer VALUES (10, 'Amit', 32, 'Delhi', 50000);  
New salary: 50000
```

```
SQL> update customer set salary=55000 where id=10;  
Old salary: 50000  
New salary: 55000  
Salary difference: 5000
```

```
SQL> delete from customer where id=10;  
Old salary: 55000  
  
1 row deleted.
```

2. Convert employee name into uppercase whenever an employee record is inserted or updated. Trigger to fire before the insert or update

```
SQL> CREATE OR REPLACE TRIGGER employee_insert_update
2  BEFORE INSERT OR UPDATE ON employee
3  FOR EACH ROW
4  BEGIN
5  -- Force employee first name to upper case
6  :NEW.first_name := UPPER(:NEW.first_name);
7  END;
8  /
```

Trigger created.

Output

```
SQL> INSERT INTO Employee
2  VALUES ('E01', 'John', 'Smith', SYSDATE, NULL, 50000, 'Delhi', 'Manager');

1 row created.

SQL> SELECT id, first_name FROM employee;

ID   FIRST_NAME
----  -
E01  RAHUL
E01  JOHN

SQL> update employee set first_name='prerana' where id='E02';

1 row updated.

SQL> SELECT id, first_name FROM employee;

ID   FIRST_NAME
----  -
E01  RAHUL
E01  JOHN
E02  PRERANA

SQL> |
```

3. Trigger before deleting a record from emp table. Trigger will insert the row to be deleted into another table and also record the user who has deleted the record

```
SQL> CREATE OR REPLACE TRIGGER employee_before_delete
2  BEFORE DELETE
3  ON employee
4  FOR EACH ROW
5  DECLARE
6  v_username VARCHAR2(30);
7  BEGIN
8  -- Get username of person performing DELETE
9  SELECT USER
10 INTO v_username
11 FROM dual;
12
```

```
13      -- Insert deleted record into audit table
14      INSERT INTO employee_audit
15      (id, salary, delete_date, deleted_by)
16      VALUES
17      (:OLD.id, :OLD.salary, SYSDATE, v_username);
18      END;
19      /
Trigger created.
```

Output:

```
SQL> delete from employee where id='E10';
1 row deleted.

SQL> select * from employee_audit;

ID          SALARY DELETE_DA DELETED_BY
-----
E10         45000  20-DEC-25  SYSTEM

SQL> |
```


LAB EXERCISE: 9

Q. Demonstration on Procedures

1) create a procedure to reverse a string

SQL> create or replace procedure ReverseOf(p_input in varchar) is

```
2   rev varchar(20):="";
3   begin
4   for i in reverse 1..length(p_input)loop
5   rev:=rev||substr(p_input,i,1);
6   end loop;
7   dbms_output.put_line('reverse string is '||rev);
8   end;
9   /
```

Output:

```
SQL> set serveroutput on
SQL> begin
2   ReverseOf('Hello');
3   end;
4   /
reverse string is olleH
```

LAB EXERCISE: 10**Q. Demonstration On Cursor**

```
SQL> set serveroutput on;
SQL> DECLARE
2     CURSOR xx IS
3     SELECT empno, ename, sal
4     FROM emp26;
5
6     a_empno emp26.empno%TYPE;
7     a_ename emp26.ename%TYPE;
8     a_sal emp26.sal%TYPE;
9
10    BEGIN
11    OPEN xx;
12
13    LOOP
14    FETCH xx INTO a_empno, a_ename, a_sal;
15    EXIT WHEN xx%NOTFOUND;
16
17        DBMS_OUTPUT.PUT_LINE(
18        a_empno || ' ' || a_ename || ' ' || a_sal
19        );
20    END LOOP;
21
22    CLOSE xx;
23    END;
24    /
```

Output

```
24    /
101 Shree 10000
102 Veena 20000
103 Om 15000
104 Prerana 15000
105 Rohini 18000

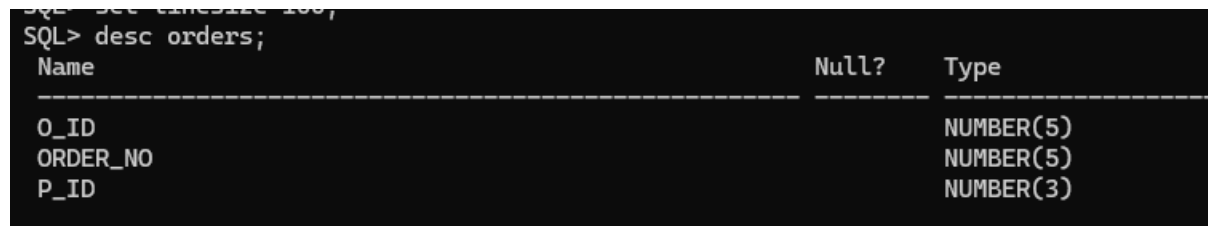
PL/SQL procedure successfully completed.

SQL> |
```

LAB EXERCISE: 11**Q. Demonstration on SQL queries to implement joins.****Joins****Left join or left outer join**

SQL> create table orders(O_Id number(5), Order_no number(5), P_Id number(3));

Table created.



The screenshot shows a terminal window with the following content:

```
SQL> desc orders;
Name                                     Null?      Type
-----
O_ID                                     NUMBER(5)
ORDER_NO                                NUMBER(5)
P_ID                                     NUMBER(3)
```

Inserting values in table:

SQL> insert into orders values(&O_ID, &Order_no, &P_Id);

Enter value for o_id: 1

Enter value for order_no: 77895

Enter value for p_id: 3

old 1: insert into orders values(&O_ID, &Order_no, &P_Id)

new 1: insert into orders values(1, 77895, 3)

1 row created.

SQL> insert into orders values(&O_ID, &Order_no, &P_Id);

Enter value for o_id: 2

Enter value for order_no: 44678

Enter value for p_id: 3

old 1: insert into orders values(&O_ID, &Order_no, &P_Id)

new 1: insert into orders values(2, 44678, 3)

1 row created.

SQL> insert into orders values(&O_ID, &Order_no, &P_Id);

Enter value for o_id: 3

Enter value for order_no: 22456

Enter value for p_id: 1

old 1: insert into orders values(&O_ID, &Order_no, &P_Id)

new 1: insert into orders values(3, 22456, 1)

1 row created.

SQL> insert into orders values(&O_ID, &Order_no, &P_Id);

Enter value for o_id: 4

Enter value for order_no: 24562

Enter value for p_id: 1

old 1: insert into orders values(&O_ID, &Order_no, &P_Id)

new 1: insert into orders values(4, 24562, 1)
1 row created.

SQL> insert into orders values(&O_ID, &Order_no, &P_Id);
Enter value for o_id: 5
Enter value for order_no: 34764
Enter value for p_id: 15
old 1: insert into orders values(&O_ID, &Order_no, &P_Id)
new 1: insert into orders values(5, 34764, 15)
1 row created.

Table Select

```
SQL> select * from orders;
```

O_ID	ORDER_NO	P_ID
1	77895	3
2	44678	3
3	22456	1
4	24562	1
5	34764	15

Table2 : persons

SQL> create table persons (P_ID number(5), Last_Name varchar2(10), First_name varchar2(15), Address varchar2(10), City varchar2(10));

Table created.

Insert into values

SQL> insert into persons values (&p_ID, &Last_Name, &First_Name,&Address, &City);
Enter value for p_id: 1
Enter value for last_name: 'Upadhye'
Enter value for first_name: 'Manasi'
Enter value for address: 'Near Math'
Enter value for city: 'Jaysingpur'
old 1: insert into persons values (&p_ID, &Last_Name, &First_Name,&Address, &City)
new 1: insert into persons values (1, 'Upadhye', 'Manasi', 'Near Math', 'Jaysingpur')
1 row created.

SQL> insert into persons values (&p_ID, &Last_Name, &First_Name,&Address, &City);
Enter value for p_id: 2
Enter value for last_name: 'Sane'
Enter value for first_name: 'Soniya'
Enter value for address: 'Ich Main'
Enter value for city: 'Nandani'
old 1: insert into persons values (&p_ID, &Last_Name, &First_Name,&Address, &City)
new 1: insert into persons values (2, 'Sane', 'Soniya', 'Ich Main', 'Nandani')
1 row created.

```
SQL> insert into persons values (&p_ID, &Last_Name, &First_Name,&Address, &City);
Enter value for p_id: 3
Enter value for last_name: 'Rane'
Enter value for first_name: 'Gomtesh'
Enter value for address: 'Kolhapur'
Enter value for city: 'Kolhapur'
old 1: insert into persons values (&p_ID, &Last_Name, &First_Name,&Address, &City)
new 1: insert into persons values (3, 'Rane', 'Gomtesh','Kolhapur', 'Kolhapur')
1 row created.
```

```
SQL> select * from persons;
```

P_ID	LAST_NAME	FIRST_NAMR	ADDRESS	CITY
1	Upadhye	Manasi	Near Math	Jaysingpur
2	Sane	Soniya	Ich Main	Nandani
3	Rane	Gomtesh	Kolhapur	Kolhapur

Left Join Syntax

```
SQL> select persons.Last_Name,persons.First_Namr,orders.Order_no from persons left join
orders on persons.p_Id = orders.P_ID order by persons.Last_Name;
```

LAST_NAME	FIRST_NAMR	ORDER_NO
Rane	Gomtesh	77895
Rane	Gomtesh	44678
Sane	Soniya	
Upadhye	Manasi	24562
Upadhye	Manasi	22456

Full Outer join

```
SQL> select * from persons;
```

```
SQL> select * from persons;
```

P_ID	LAST_NAME	FIRST_NAMR	ADDRESS	CITY
1	Upadhye	Manasi	Near Math	Jaysingpur
2	Sane	Soniya	Ich Main	Nandani
3	Rane	Gomtesh	Kolhapur	Kolhapur

```
SQL> select * from orders;
```

```
SQL> select * from orders;
```

O_ID	ORDER_NO	P_ID
1	77895	3
2	44678	3
3	22456	1
4	24562	1
5	34764	15

Full Outer Join Syntax:

SQL> select persons.Last_Name,persons.First_Namr,orders.Order_no from persons full outer join orders on persons.p_Id = orders.P_ID order by persons.Last_Name;

LAST_NAME	FIRST_NAMR	ORDER_NO
Rane	Gomtesh	77895
Rane	Gomtesh	44678
Sane	Soniya	
Upadhye	Manasi	24562
Upadhye	Manasi	22456
		34764

Right Outer join:

SQL> select persons.Last_Name,persons.First_Namr,orders.Order_no from persons right outer join orders on persons.p_Id = orders.P_ID order by persons.Last_Name;

LAST_NAME	FIRST_NAMR	ORDER_NO
Rane	Gomtesh	44678
Rane	Gomtesh	77895
Upadhye	Manasi	22456
Upadhye	Manasi	24562
		34764

Inner Join:

SQL> select persons.Last_Name,persons.First_Namr,orders.Order_no from persons inner join orders on persons.p_Id = orders.P_ID order by persons.Last_Name;

LAST_NAME	FIRST_NAMR	ORDER_NO
Rane	Gomtesh	77895
Rane	Gomtesh	44678
Upadhye	Manasi	22456
Upadhye	Manasi	24562

LAB EXERCISE: 12

Q. Demonstration on View.

Create STUDENT DETAILS TABLE

Create StudentDetails table

```
CREATE TABLE StudentDetails (  
    S_ID INT PRIMARY KEY,  
    NAME VARCHAR(15),  
    ADDRESS VARCHAR(15)  
);
```

```
SQL> desc StudentDetails;
```

Name	Null?	Type
S_ID	NOT NULL	NUMBER(5)
NAME		VARCHAR2(15)
ADDRESS		VARCHAR2(15)

```
INSERT INTO StudentDetails (S_ID, NAME, ADDRESS)
```

```
VALUES
```

```
(1, 'Kalyani', 'Kolkata'),
```

```
(2, 'Ashish', 'Durgapur'),
```

```
(3, 'Pratik', 'Delhi'),
```

```
(4, 'Dhanraj', 'Bihar'),
```

```
(5, 'Ram', 'Rajasthan');
```

```
SQL> select * from StudentDetails;
```

S_ID	NAME	ADDRESS
1	Kalyani	Kolkata
2	Ashish	Durgapur
3	Pratik	Delhi
4	Dhanraj	Bihar
5	Ram	Rajasthan

Create StudentMarks table

```
CREATE TABLE StudentMarks (  
    ID INT PRIMARY KEY,
```

```
    NAME VARCHAR(15),
```

```
    Marks INT,
```

```
    Age INT
```

);

INSERT INTO StudentMarks (ID, NAME, Marks, Age)

VALUES

(1, 'Kalyani', 90, 21),

(2, 'Suresh', 50, 20),

(3, 'Pratik', 80, 19),

(4, 'Dhanraj', 95, 21),

(5, 'Ram', 85, 18);

```
SQL> select * from StudentMarks ;
```

ID	NAME	MARKS	AGE
1	Kalyani	90	21
2	Suresh	50	20
3	Pratik	80	19
4	Dhanraj	95	21
5	Ram	85	18

CREATE VIEW DetailsView **AS**

SELECT NAME, ADDRESS

FROM StudentDetails **WHERE**

S_ID < 5;

SELECT * FROM DetailsView;

```
SQL> SELECT * FROM DetailsView;
```

NAME	ADDRESS
Kalyani	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar

Example 2: Create View From Table

CREATE VIEW StudentNames **AS**

SELECT S_ID, NAME

FROM StudentDetails

ORDER BY NAME;

SELECT * FROM StudentNames;


```
SQL> SELECT * FROM StudentNames;
```

S_ID	NAME
2	Ashish
4	Dhanraj
1	Kalyani
3	Pratik
5	Ram

Example 3: Creating View from multiple tables

CREATE VIEW MarksView AS

SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS

FROM StudentDetails, StudentMarks

WHERE StudentDetails.NAME = StudentMarks.NAME;

SELECT * FROM MarksView;

```
SQL> SELECT * FROM MarksView;
```

NAME	ADDRESS	MARKS
Kalyani	Kolkata	90
Pratik	Delhi	80
Dhanraj	Bihar	95
Ram	Rajasthan	85

DELETE VIEWS

Syntax

DROP VIEW view_name;

Example

In this example, we are deleting the View **MarksView**.

DROP VIEW MarksView;

```
SQL> DROP VIEW MarksView;
```

```
View dropped.
```

UPDATE VIEW

Syntax

UPDATE view_name

SET column1 = value1, column2 = value2...., columnN = valueN

WHERE [condition];

Example 1: Update View to Add or Replace a View Field

We can use the **CREATE OR REPLACE VIEW** statement to add or replace fields from a view.

If we want to update the view **MarksView** and add the field AGE to this View from **StudentMarks** Table

CREATE OR REPLACE VIEW MarksView AS

SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS,
StudentMarks.AGE

FROM StudentDetails, StudentMarks

WHERE StudentDetails.NAME = StudentMarks.NAME;

SELECT * FROM MarksView;

```
SQL> SELECT * FROM MarksView;
```

NAME	ADDRESS	MARKS	AGE
-----	-----	-----	-----
Kalyani	Kolkata	90	21
Pratik	Delhi	80	19
Dhanraj	Bihar	95	21
Ram	Rajasthan	85	18

Example 2: Update View to Insert a row in a view

INSERT INTO DetailsView(NAME, ADDRESS)

VALUES('Suresh', 'Gurgaon');

SELECT * FROM DetailsView;

```
SQL> SELECT * FROM DetailsView;
```

NAME	ADDRESS
-----	-----
Kalyani	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar
Ram	Rajasthan
Suresh	Gurgaon

Example 3: Deleting a row from a View

Deleting rows from a view is also as simple as deleting rows from a table. We can use the DELETE statement of SQL to delete rows from a view. Also deleting a row from a view first deletes the row from the actual table and the change is then reflected in the view.

In this example, we will delete the last row from the view DetailsView which we just added in the above example of inserting rows.

DELETE FROM DetailsView

WHERE NAME="Suresh";

SELECT * FROM DetailsView;

```
SQL> SELECT * FROM DetailsView;
```

NAME	ADDRESS
Kalyani	Kolkata
Ashish	Durgapur
Pratik	Delhi
Dhanraj	Bihar
Ram	Rajasthan

LAB EXERCISE: 13

Q. Demonstration on String Function and date function.

STRING FUNCTIONS:

Concat: CONCAT returns char1 concatenated with char2. Both char1 and char2 can be any of the datatypes

```
SQL>SELECT CONCAT('ORACLE','CORPORATION') FROM DUAL;  
ORACLECORPORATION
```

Output :

```
SQL> select concat('ORCALE','CORPORATION')from dual;  
  
CONCAT('ORCALE','  
-----  
ORCALECORPORATION
```

Lpad: LPAD returns expr1, left-padded to length n characters with the sequence of characters in expr2.

```
SQL>SELECT LPAD('ORACLE',15,'*')FROM DUAL;  
  
*****ORACLE
```

Output :

```
SQL> select LPAD('ORCALE',15,'*')from dual;  
  
LPAD('ORCALE',1  
-----  
*****ORCALE
```

Rpad: RPAD returns expr1, right-padded to length n characters with expr2, replicated as many times as necessary.

```
SQL>SELECT RPAD ('ORACLE',15,'*')FROM DUAL; ORACLE*****
```

Output :

```
SQL> select RPAD('ORCALE',15,'*')from dual;  
  
RPAD('ORCALE',1  
-----  
ORCALE*****
```

Ltrim: Returns a character expression after removing leading blanks.

SQL>SELECT LTRIM('SSMITHSS','S')FROM DUAL; MITHSS

Output :

```
SQL> select LTRIM('SSMITHSS','S')from dual;

LTRIM(
-----
MITHSS
```

Rtrim: Returns a character string after truncating all trailing blanks

SQL>SELECT RTRIM('SSMITHSS','S')FROM DUAL; SSMITH

Output :

```
SQL> select RTRIM('SSMITHSS','S')from dual;

RTRIM(
-----
SSMITH
```

Lower: Returns a character expression after converting uppercase character data to lowercase.

SQL>SELECT LOWER('DBMS')FROM DUAL;

Dbms

Output :

```
SQL> select LOWER('RDBMS')from dual;

LOWER
-----
rdbms
```

Upper: Returns a character expression with lowercase character data converted to uppercase

SQL>SELECT UPPER('dbms')FROM DUAL; DBMS

Output :

```
SQL> select UPPER('rdbms')from dual;

UPPER
-----
RDBMS
```

Length: Returns the number of characters, rather than the number of bytes, of the given string expression, excluding trailing blanks.

SQL>SELECT LENGTH('DATABASE')FROM DUAL;

8

Output :

```
SQL> select LENGTH('DATABASE')from dual;

LENGTH('DATABASE')
-----
                        8
```

Substr: Returns part of a character, binary, text, or image expression.

SQL>SELECT SUBSTR('ABCDEFGHIJ',3,4) FROM DUAL; CDEF

Output :

```
SQL> select SUBSTR('ABCDEFGHIJ',3,4)from dual;

SUBS
----
CDEF
```

Instr: The INSTR functions search string for substring.

The function returns an integer indicating the position of the character in string that is the first character of this occurrence.

SQL>SELECT INSTR('CORPORATE FLOOR','OR',3,2)FROM DUAL;

14

Output :

```
SQL> select INSTR('CORPORATE FLOOR','OR',3,2)from dual;

INSTR('CORPORATEFLOOR','OR',3,2)
-----
                        14
```

DATE FUNCTIONS:**Sysdate:**

SQL>SELECT SYSDATE FROM DUAL;

29-DEC-08

Output :

```
SQL> select SYSDATE from dual;

SYSDATE
-----
09-DEC-25
```

next_day:

SQL>SELECT NEXT_DAY(SYSDATE,'WED')FROM DUAL;

05-JAN-09

Output :

```
SQL> select NEXT_DAY(SYSDATE,'WED')from dual;

NEXT_DAY(
-----
10-DEC-25
```

add_months:

SQL>SELECT ADD_MONTHS(SYSDATE,2)FROM DUAL;

28-FEB-09

Output :

```
SQL> select ADD_MONTHS(SYSDATE,2)from dual;

ADD_MONTH
-----
09-FEB-26
```

last_day:

SQL>SELECT LAST_DAY(SYSDATE)FROM DUAL;

31-DEC-08

```
SQL> select LAST_DAY(SYSDATE)from dual;

LAST_DAY(
-----
31-DEC-25
```

months_between:

SQL>SELECT MONTHS_BETWEEN(SYSDATE,HIREDATE)FROM EMP;

4

Least:

SQL>SELECT LEAST('10-JAN-07','12-OCT-07')FROM DUAL;

10-JAN-07

Output :

```
SQL> select LEAST('12-Oct-07')from dual;

LEAST('12
-----
12-Oct-07
```

Greatest:

SQL>SELECT GREATEST('10-JAN-07','12-OCT-07')FROM DUAL;

10-JAN-07

Output :

```
SQL> select GREATEST('12-Oct-07')from dual;

GREATEST(
-----
12-Oct-07
```

Trunc:

SQL>SELECT TRUNC(SYSDATE,'DAY')FROM DUAL;

28-DEC-08

Output :

```
SQL> select TRUNC(SYSDATE,'DAY')from dual;

TRUNC(SYS
-----
07-DEC-25
```

Round:20

SQL>SELECT ROUND(SYSDATE,'DAY')FROM DUAL;

28-DEC-08

Output :

```
SQL> select ROUND(SYSDATE,'DAY')from dual;

ROUND(SYS
-----
07-DEC-25
```

to_char:

```
SQL> select to_char(sysdate, "dd\mm\yy") from dual;
```

24-mar-05.

Output :

```
SQL> select to_char(sysdate,'dd\mm\yy')from dual;

TO_CHAR(
-----
09\12\25
```

to_date:

```
SQL> select to date (sysdate, "dd\mm\yy") from dual;
```

24-mar-05.

Output :

```
SQL> select to_date(sysdate,'dd\mm\yy')from dual;

TO_DATE(S
-----
09-DEC-25
```

LAB EXERCISE: 14

Q. Demonstration on Set Operation.

SQL> select*from ln1;

```
SQL> select * from ln1;
```

LNO	BNAME	AMOUNT
1002	Airport	4001
1003	Jk Nagar	4003
1004	SkyLand	4004

SQL> select*from act;

```
SQL> select * from act;
```

ANO	BNAME	AMOUNT
12345	Contanment	20000
12346	Trichy	19000
12347	Jk Nagar	17000
12348	Jk Bagar	15000

//UNION ALL

SQL> select bname from ln1 union all select bname from act;

```
SQL> select Bname from ln1 union all select BName from act;
```

BNAME
Airport
Jk Nagar
SkyLand
Contanment
Trichy
Jk Nagar
Jk Bagar

7 rows selected.

//UNION

SQL> select bname from
ln1 union select bname from
act;

```
SQL> select Bname from ln1 union select BName from act;
```

BNAME
Airport
Contanment
Jk Bagar
Jk Nagar
SkyLand
Trichy

6 rows selected.

//INTERSECT

SQL> select bname from ln1 intersect select bname from act;

```
SQL> select Bname from ln1 intersect select BName from act;

BNAME
-----
Jk Nagar
```

//MINUS

SQL> select bname from ln1 minus select bname from act;

```
SQL> select Bname from ln1 minus select BName from act;

BNAME
-----
Airport
```

LAB EXERCISE: 15**Q. Demonstration on Index.****Index**

SQL> CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(50), Last_Name VARCHAR(50), Country VARCHAR(50), Age number(2), Phone number(10));

Table created.

SQL> desc Customer;

```
SQL> desc Customer;
Name                               Null?      Type
-----
CUSTOMER_ID                       NOT NULL   NUMBER(38)
CUSTOMER_NAME                      NULL       VARCHAR2(50)
LAST_NAME                         NULL       VARCHAR2(50)
COUNTRY                           NULL       VARCHAR2(50)
AGE                               NULL       NUMBER(2)
PHONE                             NULL       NUMBER(10)
```

SQL> INSERT INTO Customer (Customer_ID, Customer_Name, Last_Name, Country, Age, Phone)

2 Values(101,'Ram','Mane','India',25,4587941258);

1 row created.

SQL> INSERT INTO Customer (Customer_ID, Customer_Name, Last_Name, Country, Age, Phone)

2 Values(102,'Rahul','Sane','London',30,1478456745);

1 row created.

SQL> INSERT INTO Customer (Customer_ID, Customer_Name, Last_Name, Country, Age, Phone)

2 Values(103,'Sonal','John','America',24,1234567894);

1 row created.

SQL> INSERT INTO Customer (Customer_ID, Customer_Name, Last_Name, Country, Age, Phone)

2 Values(104,'Manasi','Upadhye','Singapur',24,1478523654);

1 row created.

SQL> INSERT INTO Customer (Customer_ID, Customer_Name, Last_Name, Country, Age, Phone)

2 Values(105,'Soniya','Chvan','Kashmir',28,3692547841);

1 row created.

```
SQL> select * from Customer;
```

CUSTOMER_ID	CUSTOMER_NAME	LAST_NAME	COUNTRY	AGE	PHONE
101	Ram	Mane	India	25	4587941258
102	Rahul	Sane	London	30	1478456745
103	Sonal	John	America	24	1234567894
104	Manasi	Upadhye	Singapur	24	1478523654
105	Soniya	Chvan	Kashmir	28	3692547841

Create Index

```
SQL> create index City on Customer(Country);
```

Index created.

Select table

```
SQL> select * from Customer where Country = 'India' and Age = 25;
```

CUSTOMER_ID	CUSTOMER_NAME	LAST_NAME	COUNTRY	AGE	PHONE
101	Ram	Mane	India	25	4587941258

Drop Index:

```
SQL> DROP INDEX City;
```

Index dropped.